

Universidad Politécnica de Aguascalientes

Ingeniería en Tecnologías de la Información

Tecnologías y Aplicaciones en Internet

Investigación

Frameworks de Aplicación del Lado del Cliente y del Servidor

Práctica Integradora No. 2 – Portafolio 2

Integrantes: Armando Guadarrama Jiménez
Andrei Torres Sánchez
Luis Daniel Vidales Padilla

Docente: Daniel Colunga Galván

Fecha: 1 de julio de 2026

Índice

Introducción

El desarrollo de aplicaciones web ha evolucionado de manera acelerada durante las últimas dos décadas, pasando de páginas estáticas a sistemas complejos capaces de procesar transacciones, gestionar usuarios, generar reportes y presentar interfaces dinámicas e interactivas. En este contexto, los **frameworks** han surgido como la respuesta de la industria a la necesidad de estructurar, estandarizar y acelerar el desarrollo de software.

Un framework no es simplemente una colección de funciones útiles; es un **esqueleto de aplicación completo** que impone una arquitectura, un conjunto de convenciones y un flujo de trabajo. Esto diferencia a los frameworks de las librerías: mientras una librería ofrece herramientas específicas que el desarrollador usa a su criterio, un framework define *cómo* debe construirse la aplicación.

El presente documento investiga ocho frameworks ampliamente utilizados en la industria, agrupados en dos categorías:

- **Frameworks del lado del cliente** (*frontend*): **Knockout** y **AngularJS**, que se ejecutan en el navegador y controlan la presentación e interacción con el usuario.
- **Frameworks del lado del servidor** (*backend*): **Kraken**, **Mach**, **Symfony**, **Laravel**, **Zend** y **Spring**, que se ejecutan en el servidor y gestionan la lógica de negocio, bases de datos y APIs.

Para cada framework se describen su arquitectura, lenguaje utilizado, patrón de diseño, ventajas, desventajas, casos de uso y el proceso general de construcción de aplicaciones. El documento concluye con un cuadro comparativo global y las conclusiones individuales del equipo.

1. Frameworks del Lado del Cliente

Los frameworks del lado del cliente se ejecutan dentro del navegador web del usuario. Su objetivo es gestionar la presentación, el estado de la interfaz y la comunicación con el servidor sin requerir recargas de página completa, permitiendo construir aplicaciones de una sola página (*Single Page Applications*, SPA).

1.1. Knockout

1.1.1. Descripción general

Knockout.js es una librería/micro-framework JavaScript creada por **Steve Sanderson** (Microsoft) y lanzada en julio de **2010** [?]. Está diseñada exclusivamente para implementar el patrón **MVVM** (Model-View-ViewModel) en el navegador, facilitando la sincronización automática entre los datos y la interfaz de usuario mediante un sistema de **observables** y **bindings declarativos**.

Knockout – Definición

Knockout es un framework JavaScript que implementa el patrón MVVM mediante observables y enlaces declarativos (**data-bind**), manteniendo la interfaz sincronizada automáticamente con los datos del ViewModel.

1.1.2. Arquitectura

Knockout implementa el patrón **MVVM** (Model-View-ViewModel), compuesto por tres capas [?]:

- **Model:** los datos de la aplicación provenientes del servidor (JSON, AJAX).
- **ViewModel:** objeto JavaScript que expone datos y comandos a la vista. Contiene **observables** (`ko.observable()`) que notifican cambios a la vista automáticamente.
- **View:** el HTML enriquecido con atributos **data-bind**, que declara qué propiedad del ViewModel se enlaza a cada elemento del DOM.

Listing 1: Ejemplo MVVM con Knockout.js

```
1 <!-- VIEW -->
2 <p>Pel cula: <span data-bind="text: pelicula"></span></p>
3 <input data-bind="value: pelicula" />
4 <p>Precio: <strong data-bind="text: precio"></strong></p>
5
6 <script>
7 // VIEWMODEL
8 function CineViewModel() {
9     this.pelicula = ko.observable("Inception");
10    this.precio    = ko.observable(95.00);
11 }
12
```

```
13 ko.applyBindings(new CineViewModel());  
14 </script>
```

El **sistema de dependencias** de Knockout rastrea automáticamente qué observables utiliza cada binding, actualizando solo los elementos del DOM afectados cuando un observable cambia (**actualizaciones granulares**).

1.1.3. Ventajas

- **Ligero:** sin dependencias externas, ~60 KB minificado [?].
- **Integración sencilla con HTML existente:** se añade sobre páginas ya hechas sin reestructurar el proyecto.
- **Two-way data binding automático:** los cambios en el ViewModel se reflejan en la vista y viceversa sin código adicional.
- **Observables computados:** valores derivados que se actualizan automáticamente (`ko.computed()`).
- **Curva de aprendizaje baja:** solo requiere conocer JavaScript básico y HTML.

1.1.4. Desventajas

- **Ecosistema pequeño:** comunidad y mercado laboral reducidos frente a Angular, React o Vue.
- **No apto para aplicaciones grandes:** carece de router oficial, gestión de estado avanzada ni CLI.
- **Rendimiento en listas grandes:** al usar `foreach` con miles de elementos, el DOM puede saturarse.
- **Mantenimiento reducido:** el ritmo de actualizaciones es bajo comparado con frameworks modernos.
- **Sin separación clara por componentes:** no tiene un sistema de componentes tan sólido como Vue o React.

1.1.5. Casos de uso

- Formularios complejos con validación reactiva (ej. formularios de pago, configuradores de productos).
- Añadir interactividad a aplicaciones legadas (páginas renderizadas en servidor con PHP/Razor).
- Aplicaciones de gestión empresarial interna con tablas y filtros dinámicos.
- Dashboards de lectura de datos que se actualizan por polling AJAX.

1.1.6. Proceso de construcción de aplicaciones con Knockout

1. **Incluir la librería** vía CDN o npm: `<script src="knockout.min.js"></script>`.
2. **Definir el Model:** estructuras de datos que llegan del servidor (vía `fetch/$.ajax`).
3. **Crear el ViewModel:** clase o función JS con `ko.observable()` para cada dato reactivo y `ko.computed()` para valores derivados.
4. **Diseñar la View:** HTML con atributos `data-bind` (`text`, `value`, `click`, `foreach`, `visible`, `css`, etc.).
5. **Activar Knockout:** llamar `ko.applyBindings(new ViewModel())` una sola vez al cargar la página.
6. **Comunicación con el servidor:** usar `fetch()` o jQuery AJAX para obtener/enviar datos; actualizar los observables con la respuesta.

1.1.7. Ficha técnica

Atributo	Valor
Tipo	Micro-framework / Librería JS
Patrón	MVVM
Lenguaje	JavaScript
Versión actual	3.5.1 (2019)
Creador	Steve Sanderson – Microsoft (2010)
Licencia	MIT
Sitio oficial	knockoutjs.com

1.2. AngularJS

1.2.1. Descripción general

AngularJS (también llamado Angular 1) es un framework JavaScript de código abierto creado por **Miško Hevery** en **Google** y lanzado en **2010** [?]. Fue el primer framework completo para construir SPAs en el navegador, introduciendo conceptos como la inyección de dependencias del lado del cliente, las directivas HTML extendidas y el two-way data binding masivo. En 2022, Google declaró el fin oficial del soporte de AngularJS [?]; su sucesor es **Angular** (v2+, reescrito en TypeScript).

AngularJS – Definición

AngularJS es un framework JavaScript completo que extiende HTML con directivas propias, implementa el patrón MVC en el navegador y provee two-way data binding, inyección de dependencias y un sistema de módulos para construir SPAs.

1.2.2. Arquitectura

AngularJS implementa una arquitectura **MVC del lado del cliente** con los siguientes elementos [?]:

- **Module:** unidad de organización del código (`angular.module()`); agrupa controladores, servicios y directivas.
- **Controller:** define la lógica de una vista y expone datos al `$scope`.
- **\$scope:** objeto que actúa como puente entre el Controller y la View; los cambios en él se reflejan automáticamente en el HTML.
- **View:** HTML enriquecido con directivas (`ng-model`, `ng-repeat`, `ng-if`, `ng-click`, etc.).
- **Service/Factory:** lógica de negocio reutilizable (ej. comunicación HTTP con `$http`).
- **Directiva:** componente HTML personalizado que encapsula comportamiento y presentación.
- **Dependency Injection (DI):** el framework inyecta automáticamente las dependencias declaradas en los controladores y servicios.

Listing 2: Ejemplo MVC con AngularJS

```
1 <div ng-app="CineApp" ng-controller="PelículaCtrl">
2   <h2>{{ pelicula.nombre }}</h2>
3   <ul>
4     <li ng-repeat="h in horarios">{{ h }}</li>
5   </ul>
6   <button ng-click="comprar()">Comprar boleto</button>
7 </div>
8
9 <script>
10 angular.module('CineApp', [])
11   .controller('PelículaCtrl', ['$scope', '$http', function($scope,
12     $http) {
13     $scope.pelicula = { nombre: 'Inception', precio: 95 };
14     $scope.horarios = ['13:00', '16:30', '20:00'];
15     $scope.comprar = function() { alert('Comprando...'); };
16   }]);
17 </script>
```

1.2.3. Ventajas

- **Framework completo:** router, HTTP, formularios, animaciones y DI incluidos sin librerías externas.
- **Two-way data binding:** sincronización automática DOM ↔ `$scope`.
- **Inyección de dependencias nativa:** facilita la modularidad y el testing unitario.

- **Directivas:** permiten crear componentes HTML reutilizables y semánticamente expresivos.
- **Gran adopción histórica:** amplísima documentación y recursos de aprendizaje.

1.2.4. Desventajas

- **Sin soporte oficial desde 2022:** no recibe parches de seguridad ni nuevas funcionalidades [?].
- **Rendimiento en aplicaciones grandes:** el ciclo de *digest* (revisión de `$scope`) puede volverse lento con muchos bindings.
- **Curva de aprendizaje pronunciada:** conceptos como `$scope`, `$apply`, directivas y DI son complejos para principiantes.
- **No es compatible con Angular 2+:** la migración es prácticamente una reescritura completa.
- **Legacy:** proyectos nuevos deberían usar Angular, React o Vue.

1.2.5. Casos de uso

- Mantenimiento de aplicaciones empresariales legadas construidas con AngularJS (período 2012–2020).
- SPAs de tamaño medio con formularios complejos (portales internos, dashboards administrativos).
- Aplicaciones de Google de la era 2012–2016 (Google Fiber, DoubleClick, etc.).

1.2.6. Proceso de construcción de aplicaciones con AngularJS

1. **Incluir AngularJS** vía CDN o Bower: `<script src=.^angular.min.js></script>`.
2. **Definir el módulo principal:** `angular.module('App', ['ngRoute'])`.
3. **Configurar el router** (`$routeProvider`) para mapear URLs a vistas parciales y controladores.
4. **Crear controladores** que expongan datos y funciones al `$scope`.
5. **Crear servicios** para la comunicación HTTP (`$http.get/post`) y la lógica de negocio.
6. **Diseñar las vistas HTML** con directivas AngularJS (`ng-*`).
7. **Crear directivas propias** para componentes reutilizables (ej. tarjeta de película, mapa de asientos).
8. **Pruebas unitarias** con Karma + Jasmine, aprovechando la DI para inyectar mocks.

1.2.7. Ficha técnica

Atributo	Valor
Tipo	Framework JS completo (SPA)
Patrón	MVC del lado del cliente
Lenguaje	JavaScript
Versión final	1.8.3 (2022, soporte terminado)
Creador	Misko Hevery – Google (2010)
Licencia	MIT
Sitio oficial	angularjs.org

2. Frameworks del Lado del Servidor

Los frameworks del lado del servidor se ejecutan en el servidor web y se encargan de recibir peticiones HTTP, aplicar la lógica de negocio, interactuar con la base de datos y devolver respuestas al cliente. Son la base de cualquier API REST, aplicación web o sistema de gestión backend.

2.1. Kraken.js

2.1.1. Descripción general

Kraken.js es un framework de Node.js creado por **PayPal** en **2013** [?]. Está construido sobre **Express** y añade una capa de **convención sobre configuración**, seguridad por defecto y una estructura de proyecto estandarizada, pensada para aplicaciones empresariales de gran escala en Node.js.

Kraken.js – Definición

Kraken es un framework empresarial de Node.js construido sobre Express que agrega convenciones de proyecto, seguridad robusta, configuración por entorno y soporte de internacionalización (i18n) desde el arranque.

2.1.2. Arquitectura

Kraken sigue una arquitectura en capas sobre Express [?]:

- **Bootstrapping:** inicialización del servidor Express con middlewares preconfigurados (seguridad, compresión, internacionalización).
- **Configuración por entorno:** archivos JSON separados por entorno (`development.json`, `production.json`) cargados automáticamente por el módulo **confit**.
- **Enrutamiento automático:** las rutas se cargan desde una carpeta `/controllers` siguiendo convenciones de nombres de archivo.
- **Middlewares de seguridad:** integración de **lusca** (protección CSRF, XSS, CSP, HSTS).
- **Templating:** soporte para **Dust.js** (motor de plantillas de LinkedIn/PayPal).

2.1.3. Lenguaje, patrón de diseño, ventajas y desventajas

Atributo	Descripción
Lenguaje	JavaScript (Node.js)
Patrón	MVC + Convención sobre Configuración
Ventajas	Seguridad integrada (lusca), estructura estándar para equipos grandes, configuración por entorno, apoyo de PayPal

Atributo	Descripción
Desventajas	Comunidad pequeña, poco activo desde 2017, limitado a Node.js/Express, escasa documentación actualizada
Versión actual	2.3.x (github.com/krakenjs/kraken-js)

2.1.4. Casos de uso

Aplicaciones empresariales Node.js con equipos grandes que requieren estructura predecible; fue utilizado internamente por PayPal para migrar sus aplicaciones Java a Node.js [?].

2.1.5. Proceso de construcción con Kraken

1. Instalar con `npm install -g generator-kraken` y generar el proyecto con `yo kraken`.
2. Configurar entornos en `/config/{environment}.json`.
3. Definir rutas en `/controllers` (Kraken las carga automáticamente por convención).
4. Implementar modelos en `/models` con el ORM de preferencia (Sequelize, Mongoose).
5. Configurar plantillas Dust.js en `/public/templates`.
6. Ejecutar con `npm start`; el middleware lusca activa la protección de seguridad automáticamente.

2.2. Mach

2.2.1. Descripción general

Mach es un framework web minimalista para **Node.js** inspirado en **Rack** (Ruby) y **WSGI** (Python) [?]. Su filosofía central es que una aplicación web no es más que una **función** que recibe una petición y devuelve una promesa de respuesta. Esto lo hace extremadamente composable mediante una pila de **middleware funcionales**.

Mach – Definición

Mach es un framework Node.js basado en el concepto de que una aplicación web es una función pura: recibe un objeto de conexión y devuelve una promesa de respuesta, permitiendo componer comportamientos mediante middleware funcional.

2.2.2. Arquitectura y características

- **Función como aplicación:** toda la aplicación (o middleware) es una función `app(conn) => Promise<response>`.
- **Pila de middleware:** se construye apilando funciones con `mach.stack()`.
- **Basado en promesas:** todas las operaciones asíncronas se manejan con Promises nativas.

- **Enrutamiento:** mediante `mach.router()` que mapea patrones URL a funciones.

Atributo	Descripción
Lenguaje	JavaScript (Node.js)
Patrón	Middleware funcional (inspirado en Rack/WSGI)
Ventajas	Extremadamente ligero, composable, testeable (funciones puras), sin magia ni convenciones implícitas
Desventajas	Comunidad casi inexistente, repositorio inactivo (último commit 2016), sin ecosistema de plugins, no apto para producción moderna
Versión actual	0.14.x (github.com/mjackson/mach – archivado)

2.2.3. Casos de uso

Prototipos rápidos de servicios HTTP, microservicios muy pequeños, proyectos académicos de demostración de arquitecturas middleware funcionales.

2.2.4. Proceso de construcción con Mach

1. Instalar: `npm install mach`.
2. Crear la función de aplicación: `function app(conn) { return conn.text(200, 'Hola'); }`.
3. Componer middleware con `mach.stack()`: logger, autenticación, enrutador.
4. Definir rutas con `mach.router()` mapeando métodos HTTP a funciones.
5. Levantar el servidor: `mach.serve(app, 4000)`.

2.3. Symfony

2.3.1. Descripción general

Symfony es un framework PHP de código abierto creado por **Fabien Potencier** (Sensio-Labs) y lanzado en **2005** [?]. Es uno de los frameworks PHP más maduros y robustos; su arquitectura basada en **componentes reutilizables** ha influenciado profundamente el ecosistema PHP: tanto **Laravel** como **Drupal**, **Magento** y **phpBB** utilizan componentes de Symfony internamente.

Symfony – Definición

Symfony es un framework PHP empresarial basado en componentes desacoplados y reutilizables, que implementa el patrón MVC y sigue los estándares PHP-FIG (PSR) para facilitar la interoperabilidad entre proyectos.

2.3.2. Arquitectura

Symfony sigue una arquitectura **MVC orientada a componentes** [?]:

- **HttpKernel:** núcleo que maneja el ciclo petición/respuesta mediante eventos (`KernelEvents`).
- **Router:** mapea URLs a controladores; configuración en YAML, XML, anotaciones PHP o atributos PHP 8.
- **Controlador:** clase PHP que recibe una `Request` y devuelve una `Response`.
- **Doctrine ORM:** mapeador objeto-relacional para interactuar con la BD.
- **Twig:** motor de plantillas para las vistas.
- **DependencyInjection Container:** gestiona servicios y sus dependencias.
- **EventDispatcher:** arquitectura de eventos que permite extender el framework sin modificar su núcleo.

Atributo	Descripción
Lenguaje	PHP (≥ 8.1)
Patrón	MVC + Componentes + Event-Driven
Versión actual	7.x (LTS: 6.4 hasta nov 2027)
Ventajas	Madurez y estabilidad (20 años), componentes reutilizables, excelente para proyectos grandes, documentación exhaustiva, ecosistema de bundles
Desventajas	Curva de aprendizaje pronunciada, configuración verbosa, más lento que microframeworks PHP, no es la mejor opción para proyectos pequeños

2.3.3. Casos de uso

E-commerce de alto tráfico, plataformas SaaS empresariales, CMS (Drupal 8+), portales gubernamentales, APIs REST de alta complejidad.

2.3.4. Proceso de construcción con Symfony

1. Crear proyecto: `composer create-project symfony/skeleton nombre-proyecto`.
2. Instalar paquetes según necesidad: `composer require orm twig security maker`.
3. Definir entidades (modelos) con atributos PHP 8: `#[ORM\Entity]`, `#[ORM\Column]`.
4. Generar migraciones: `php bin/console doctrine:migrations:generate` y ejecutar: `doctrine:migrations:migrate`.
5. Crear controladores con `php bin/console make:controller NombreController`.
6. Configurar rutas mediante atributos PHP 8: `#[Route('/api/peliculas', name: 'peliculas_index')]`.
7. Diseñar vistas Twig en `templates/`.

8. Configurar seguridad en `config/packages/security.yaml`.
9. Levantar el servidor: `symfony server:start`.

2.4. Laravel

2.4.1. Descripción general

Laravel es el framework PHP más popular del mundo en la actualidad [?]. Fue creado por **Taylor Otwell** y lanzado en **2011** [?], con el objetivo de ofrecer una alternativa más elegante y expresiva a CodeIgniter. Su filosofía se resume en hacer que el desarrollo web sea una experiencia placentera sin sacrificar la funcionalidad.

Laravel – Definición

Laravel es un framework PHP con sintaxis expresiva y elegante que implementa el patrón MVC, incluye el ORM Eloquent (Active Record), el motor de plantillas Blade, la CLI Artisan y un ecosistema completo (Sanctum, Horizon, Telescope) para el desarrollo moderno.

2.4.2. Arquitectura

- **Patrón MVC** claro: `app/Models`, `app/Http/Controllers`, `resources/views`.
- **Eloquent ORM (Active Record)**: cada modelo representa una tabla; las relaciones se definen como métodos (`hasMany`, `belongsTo`, `belongsToMany`).
- **Blade**: motor de plantillas con herencia de layouts, componentes y directivas (`@if`, `@foreach`, `@extends`).
- **Artisan**: CLI completa para generar código, ejecutar migraciones, colas, etc.
- **Service Container**: IoC container para inyección de dependencias.
- **Middleware**: para autenticación, throttling, CORS, etc.
- **Migrations**: control de versiones del esquema de BD en PHP.

Atributo	Descripción
Lenguaje	PHP (≥ 8.2)
Patrón	MVC + Active Record (Eloquent)
Versión actual	11.x (2024)
Ventajas	Sintaxis elegante, Eloquent ORM muy intuitivo, ecosistema rico (Forge, Vapor, Sanctum, Jetstream), gran comunidad, Artisan CLI potente, excelente documentación
Desventajas	Eloquent puede generar queries ineficientes si no se optimiza, actualizaciones frecuentes, opinionado (difícil salirse de las convenciones), no el más performante en benchmarks puros

2.4.3. Casos de uso

SaaS, e-commerce, APIs REST, portales de gestión, proyectos de startups que necesitan desarrollar rápido sin sacrificar calidad; ampliamente usado por agencias digitales.

2.4.4. Proceso de construcción con Laravel

1. Crear proyecto: `composer create-project laravel/laravel nombre-proyecto`.
2. Configurar `.env`: cadena de conexión a BD, clave de aplicación, etc.
3. Definir modelos con Eloquent: `php artisan make:model Pelicula -m` (genera modelo + migración).
4. Escribir migraciones y ejecutar: `php artisan migrate --seed`.
5. Crear controladores API: `php artisan make:controller PeliculaController --api`.
6. Definir rutas en `routes/api.php`: `Route::apiResource('peliculas', PeliculaController::class)`.
7. Implementar autenticación con Laravel Sanctum o Passport (tokens API).
8. Levantar servidor: `php artisan serve`.

2.5. Zend Framework (Laminas)

2.5.1. Descripción general

Zend Framework fue creado por **Zend Technologies** (fundadores de PHP: Andi Gutmans y Zeev Suraski) y lanzado en **2006** [?]. En 2019, el proyecto fue transferido a la **Linux Foundation** y renombrado a **Laminas Project**, que es su nombre actual [?]. Es el framework PHP más orientado al desarrollo empresarial de nivel corporativo.

Zend / Laminas – Definición

Zend Framework (ahora Laminas) es un framework PHP modular y orientado a objetos para aplicaciones empresariales de alta complejidad, que provee componentes altamente desacoplados y un ServiceManager para inyección de dependencias.

2.5.2. Arquitectura

- **Arquitectura MVC basada en módulos:** cada módulo es una unidad autocontenida con sus propios controladores, vistas, modelos y configuración.
- **ServiceManager:** contenedor de IoC para gestión de dependencias; equivalente al Service Container de Laravel pero más explícito.
- **EventManager:** sistema de eventos que desacopla los componentes del sistema.
- **Doctrine:** se integra típicamente con Doctrine ORM para la capa de datos.
- **laminas-view:** sistema de vistas con helpers y layouts.

Atributo	Descripción
Lenguaje	PHP (≥ 8.1)
Patrón	MVC + Módulos + IoC + EventManager
Versión actual	Laminas MVC 3.x (2024)
Ventajas	Nivel empresarial, altamente modular (se pueden usar componentes individuales), amplia adopción en banca, salud e instituciones gubernamentales, respaldado por Linux Foundation
Desventajas	Curva de aprendizaje muy alta, configuración extremadamente verbosa, comunidad más pequeña que Symfony/Laravel, menos recursos de aprendizaje disponibles

2.5.3. Casos de uso

Banca en línea, sistemas de salud, portales gubernamentales, aplicaciones de seguros; empresas como IBM, Adobe y Sony han utilizado Zend Framework en sus plataformas.

2.5.4. Proceso de construcción con Laminas

1. Crear proyecto: `composer create-project laminas/mvc-skeleton nombre-proyecto`.
2. Configurar módulos en `config/modules.config.php`.
3. Registrar servicios en el `ServiceManager` mediante archivos `module.config.php` de cada módulo.
4. Definir rutas en la configuración del módulo (`router` $\rightarrow$ `routes`).
5. Crear controladores que extienden `AbstractActionController`.
6. Integrar Doctrine para modelos y migraciones.
7. Ejecutar con el servidor integrado de PHP o un servidor Apache/Nginx.

2.6. Spring Framework

2.6.1. Descripción general

Spring es el framework Java más utilizado en el mundo para aplicaciones empresariales [?]. Fue creado por **Rod Johnson** y publicado por primera vez en **2002** [?]. Introdujo el concepto de **Inversión de Control** (IoC) y **Programación Orientada a Aspectos** (AOP) en el ecosistema Java, simplificando drásticamente el desarrollo empresarial frente al pesado modelo EJB de la época. **Spring Boot** (2014) simplificó aún más la configuración con el principio de *convention over configuration*.

Spring – Definición

Spring es un framework Java empresarial que implementa Inversión de Control (IoC) y Programación Orientada a Aspectos (AOP). Spring Boot agrega autoconfiguración y un servidor embebido para crear APIs y microservicios con mínima configuración.

2.6.2. Arquitectura

Spring sigue una arquitectura modular de capas [?]:

- **IoC Container:** núcleo del framework; gestiona la creación e inyección de beans (objetos administrados por Spring) mediante `@Component`, `@Service`, `@Repository`, `@Controller`.
- **Spring MVC:** capa web; `DispatcherServlet` recibe peticiones y las delega a `@RestController`.
- **Spring Data JPA:** abstracción sobre JPA/Hibernate para el acceso a datos con repositorios declarativos.
- **Spring Security:** autenticación y autorización (JWT, OAuth2, LDAP).
- **Spring Boot Actuator:** endpoints de monitoreo y salud de la aplicación.
- **AOP (Aspect Oriented Programming):** permite agregar comportamientos transversales (logging, transacciones, seguridad) sin modificar el código de negocio.

Listing 3: Endpoint REST con Spring Boot

```

1  @RestController
2  @RequestMapping("/api/peliculas")
3  public class PeliculaController {
4
5      @Autowired
6      private PeliculaService peliculaService;
7
8      @GetMapping
9      public ResponseEntity<List<Pelicula>> getAll() {
10         return ResponseEntity.ok(peliculaService.findAll());
11     }
12
13     @PostMapping
14     public ResponseEntity<Pelicula> create(@RequestBody Pelicula p) {
15         return ResponseEntity.status(201).body(peliculaService.save(p));
16     }
17 }
```

Atributo	Descripción
Lenguaje	Java (también Kotlin, Groovy)
Patrón	IoC + DI + MVC + AOP

Atributo	Descripción
Versión actual	Spring 6.x / Spring Boot 3.x (2024)
Ventajas	Estabilidad empresarial probada en +20 años, ecosistema masivo, seguridad robusta, escalabilidad para microservicios, soporte cloud nativo (Spring Cloud), respaldo de VMware/Broadcom
Desventajas	Curva de aprendizaje alta (aún con Boot), aplicaciones más pesadas en memoria, tiempo de arranque mayor (aunque mejoró con GraalVM), verbosidad de Java

2.6.3. Casos de uso

APIs REST empresariales, microservicios en la nube, sistemas bancarios y financieros, plataformas de e-commerce de alto tráfico (Alibaba, Netflix, Amazon usan Spring internamente).

2.6.4. Proceso de construcción con Spring Boot

1. Generar proyecto en <https://start.spring.io> seleccionando dependencias (Web, JPA, Security, MySQL Driver).
2. Configurar `application.properties`: URL de BD, puerto, JWT secret.
3. Definir entidades JPA con `@Entity`, `@Table`, `@Column`, relaciones con `@OneToMany`, `@ManyToOne`.
4. Crear repositorios con `interface XRepository extends JpaRepository<X, Long>`.
5. Implementar servicios con `@Service` que contienen la lógica de negocio.
6. Crear controladores con `@RestController` y mapear endpoints con `@GetMapping`, `@PostMapping`, etc.
7. Configurar Spring Security para JWT: filtro de autenticación, `UserDetailsService`, `SecurityFilterChain`.
8. Ejecutar con `./mvnw spring-boot:run` o `java -jar app.jar`.

3. Cuadros Comparativos

3.1. Comparación de frameworks del lado del cliente

Criterio	Knockout	AngularJS
Tipo	Micro-framework / librería JS	Framework SPA completo
Patrón	MVVM	MVC del cliente
Lenguaje	JavaScript	JavaScript
Two-way binding	Sí (mediante observables)	Sí (mediante \$scope / digest)
Tamaño	~60 KB	~160 KB
Curva de aprendizaje	Baja	Media-Alta
Ecosistema	Pequeño	Grande (aunque legado)
Router oficial	No	Sí (ngRoute)
DI nativa	No	Sí
Estado del proyecto	Mantenimiento mínimo	Sin soporte desde 2022
Ideal para	Páginas con formularios complejos, apps legadas	SPAs medianas, proyectos pre-2020

Tabla 9: Comparación: Knockout vs AngularJS.

3.2. Comparación de frameworks del lado del servidor

Criterio	Kraken	Mach	Symfony	Laravel	Zend/Laragon	Spring
Lenguaje	JS/Node	JS/Node	PHP	PHP	PHP	Java
Patrón	MVC	Middleware	MVC+Comp.	MVC+AR	MVC+Mod.	IOC+MVC+AOP
Nivel	Empresa	Micro	Empresa	Medio-Alto	Empresa	Empresa
Madurez	Media	Baja	Alta	Alta	Muy alta	Muy alta
Comunidad	Pequeña	Mínima	Grande	Muy grande	Media	Muy grande
ORM	Cualquiera	Ninguno	Doctrine	Eloquent	Doctrine	Hibernate/JPA
Seguridad	lusca	Manual	Componente	Sanctum	Módulo	Spring Security
Vigencia	Baja	Muy baja	Alta	Muy alta	Media	Muy alta

Tabla 10: Comparación de los seis frameworks del lado del servidor.

3.3. Comparación global: cliente vs. servidor

Criterio	Frameworks del cliente	Frameworks del servidor
Dónde se ejecutan	En el navegador del usuario	En el servidor web
Lenguaje típico	JavaScript	JS, PHP, Java, Python, Ruby, Go
Propósito principal	Interfaz de usuario, reactividad, UX	Lógica de negocio, BD, seguridad, APIs
Comunicación	Con el servidor vía HTTP/WebSocket	Con el cliente vía HTTP; con BD directamente
Estado	Gestionado en memoria del navegador	Gestionado en BD, sesiones o tokens
Seguridad crítica	Validaciones de UX (siempre revalidar en servidor)	Autenticación, autorización, cifrado de datos
Patrones comunes	MVC, MVVM, Flux, Component-based	MVC, REST, Microservicios, Event-driven
Ejemplos actuales	Angular, React, Vue, Svelte	Express, Django, Laravel, Spring Boot

Tabla 11: Comparación general entre frameworks de cliente y servidor.

Conclusiones

Conclusión 1 – Armando Guadarrama Jiménez

El análisis de Knockout y AngularJS muestra que incluso los frameworks que en su momento fueron revolucionarios pueden quedar obsoletos rápidamente en un ecosistema tan dinámico como el JavaScript. La lección más importante es que los conceptos detrás de ellos (MVVM, two-way binding, inyección de dependencias) no desaparecieron: migraron y evolucionaron hacia Angular 2+, React y Vue. Conocer estos frameworks no es solo historia; es entender los problemas que los frameworks modernos resuelven y por qué los resuelven de la manera en que lo hacen. Para nuestro proyecto, esta investigación refuerza la decisión de usar Vanilla JS + Fetch en el frontend: al ser un sistema relativamente sencillo, agregar un framework completo añadiría complejidad sin beneficio real.

[Desarrollar con reflexión personal y enlace al proyecto.]

Conclusión 2 – Andrei Torres Sánchez

La diversidad de frameworks del lado del servidor refleja que no existe una solución universal: Kraken y Mach son ideales para entornos Node.js ligeros o experimentales; Symfony y Zend/Laminas dominan el espacio PHP empresarial de alta complejidad; Laravel conquistó el mercado de desarrollo ágil en PHP; y Spring sigue siendo el estándar de facto en Java para aplicaciones críticas de misión. Lo que todos comparten es que resuelven el mismo problema fundamental: abstraer el manejo de peticiones HTTP, la interacción con bases de datos y la organización del código. Para el backend de nuestro proyecto, Express con Node.js es la elección correcta: suficientemente estructurado con el patrón MVC que adoptamos, pero sin la sobrecarga de un framework empresarial completo para un sistema universitario.

[Desarrollar con reflexión personal y enlace al proyecto.]

Conclusión 3 – Luis Daniel Vidales Padilla

Una de las reflexiones más valiosas de esta investigación es que el proceso de construcción de aplicaciones con cualquier framework —sea del cliente o del servidor— sigue un patrón consistente: definir el modelo de datos, establecer las rutas, implementar la lógica de negocio, conectar la presentación y configurar la seguridad. Los frameworks difieren en cómo automatizan o convencionan cada uno de estos pasos, pero el flujo conceptual es el mismo. Esto confirma que comprender la arquitectura de software importa más que memorizar la sintaxis de un framework específico: quien entiende MVC, REST y las capas de una aplicación puede migrar entre frameworks con relativa

facilidad, adaptándose a las necesidades de cada proyecto.
[Desarrollar con reflexión personal y enlace al proyecto.]

Referencias

- [1] Sanderson, S. (2010). *Knockout: Simplify dynamic JavaScript UIs with the Model-View-View Model (MVVM) pattern*. Recuperado de <https://knockoutjs.com/>
- [2] Knockout.js Team. (2019). *Knockout documentation – Introduction*. Recuperado de <https://knockoutjs.com/documentation/introduction.html>
- [3] Google Inc. (2010). *AngularJS – Superheroic JavaScript MVW Framework*. Recuperado de <https://angularjs.org/>
- [4] Google Inc. (2022). *AngularJS Developer Guide – Conceptual Overview*. Recuperado de <https://docs.angularjs.org/guide/concepts>
- [5] Google Inc. (2021). *Stable AngularJS and Long Term Support*. Recuperado de <https://blog.angular.io/stable-angularjs-and-long-term-support-7e077635ee9c>
- [6] PayPal Engineering. (2013). *krakenjs/kraken-js: An express-based Node.js web application bootstrapping module*. GitHub. Recuperado de <https://github.com/krakenjs/kraken-js>
- [7] Pai, J. (2013). *Node.js at PayPal*. PayPal Engineering Blog. Recuperado de <https://medium.com/paypal-tech/node-js-at-paypal-4e2d1d08ce4f>
- [8] Jackson, M. (2014). *mjson/mach: HTTP for JavaScript*. GitHub. Recuperado de <https://github.com/mjson/mach>
- [9] SensioLabs. (2005). *Symfony, High Performance PHP Framework for Web Development*. Recuperado de <https://symfony.com/>
- [10] SensioLabs. (2024). *Symfony Documentation*. Recuperado de <https://symfony.com/doc/current/index.html>
- [11] Otwell, T. (2011). *Laravel – The PHP Framework For Web Artisans*. Recuperado de <https://laravel.com/>
- [12] Zend Technologies. (2006). *Zend Framework*. Recuperado de <https://framework.zend.com/>
- [13] The Linux Foundation. (2019). *Laminas Project – Enterprise-Ready PHP Components and MVC Framework*. Recuperado de <https://getlaminas.org/>
- [14] VMware Inc. (2024). *Spring Framework – Overview*. Recuperado de <https://spring.io/projects/spring-framework>
- [15] VMware Inc. (2024). *Spring Framework 6.x Reference Documentation*. Recuperado de <https://docs.spring.io/spring-framework/reference/>
- [16] Johnson, R. (2002). *Expert One-on-One J2EE Design and Development*. Wrox Press. ISBN: 978-0-7645-4385-2.
- [17] W3Techs. (2024). *Usage statistics of server-side programming languages for websites*. Recuperado de https://w3techs.com/technologies/overview/programming_language